

Technical Assessment

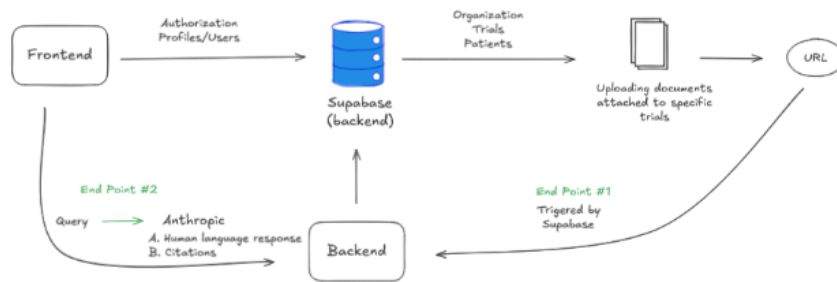
Jinpeng Zhang

October 2025

1 Overview

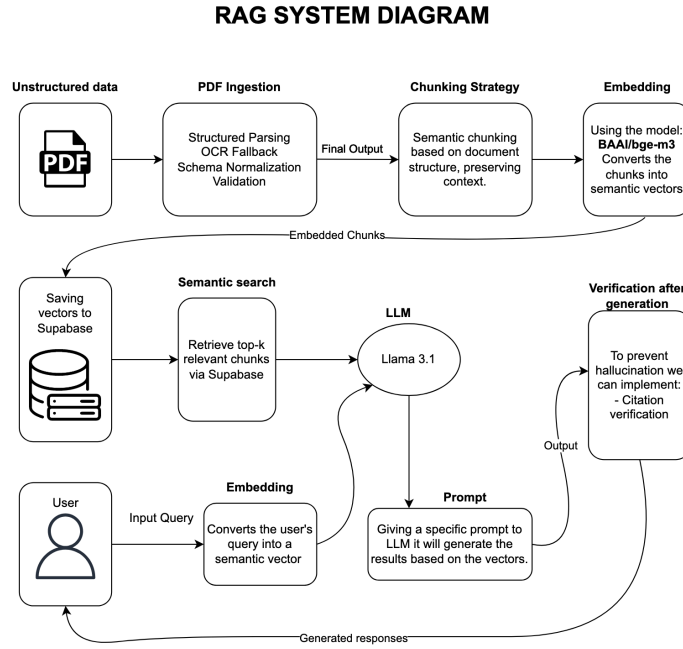
Design and describe a Retrieval-Augmented Generation (RAG) system that takes a query from a user and returns a human-language answer with accurate citations from stored PDF documents. The final outcome of the process should look like this:

Query → Human-language Response + Citations (highlighted snippets)



2 System Architecture

2.1 Detailed RAG System Diagram



2.2 PDF Ingestion

Extracting data from PDF can be really challenging as it presents unstructured data, in order to overcome this obstacle the information needs to go through a bit of processing.

Data extraction

In this specific example if we were to develop a system based on python we could use countless tools for data extraction since there are already many libraries available for such job. However in this case it is possible to use more than one to make the data more accurate:

- **1. Structured Parsing:** Use PyMuPDF4LLM as the primary parser. This converts digitally-native PDF content directly into structured Mark-down or JSON, preserving the document's hierarchy. PyMuPDF4LLM's goal is to get semantic structure (headers, lists, tables).

- **2. OCR (Optical character recognition) Fallback:** For scanned pages or non-extractable text, the images are routed to a specialized OCR engine. Pytesseract will be used in this example.
- **3. Schema Normalization:** The structured output (e.g., Markdown) is then converted into a final, standardized schema (like JSON). This step uses NLP models to map extracted text to specific data fields.
- **4. Validation:** Key sections (e.g., "Diagnosis," "Medications") are cross-checked using `regex` for known patterns and NLP models for semantic verification.
- **5. Final Result:** The output is a structured, machine-readable data object, stored with rich metadata (page, source file, section type) to enable intelligent chunking and verifiable citations.

2.3 Chunking strategy

After successfully extracting the raw data from the PDF file it will be a structured text that will then go through a chunking process. However there are many chunking strategies and we need to choose the most suitable one in the context of medical information.

However, before deciding the chunking strategy, it is fundamental to determine the chunk size, which is dictated by the chosen embedding model. For this system, we have selected BAAI/bge-m3, a state-of-the-art model with a massive 8192-token context window.

This large context window is a significant advantage for our medical use case. Instead of being limited to small, arbitrary chunks, we can create much larger, more contextually rich chunks. The best chunking strategy is **semantic chunking**, where chunks are defined by the logical structure of the document (e.g., entire paragraphs or document sections like "Methods," "Results," or "Conclusion"). This preserves the full, meaningful context, which is critical for medical accuracy.

Thus, chunking is not just about staying within token limits, but also:

- Preserving complete medical concepts and context.
- Improving the quality of the resulting embedding.
- Optimizing retrieval performance by reducing noise.

2.4 Embedding

Model Selection

For this system, we will use BAAI/bge-m3. This is a flagship, high-performance, open-source model that consistently ranks at the top of the MTEB (Massive Text Embedding Benchmark) leaderboard. It is the ideal choice for this system for three main reasons:

- **Performance:** It provides state-of-the-art accuracy in semantic retrieval.
- **Large Context Window:** It supports an 8192-token input limit. Preserving full context and reducing the risk of splitting a critical piece of information across multiple, smaller chunks.
- **Dimensions:** It converts each text chunk into a 1024-dimension vector.

Embedding Process

The process is straightforward:

1. **Input:** Each structured text chunk (up to 8192 tokens) is taken as input.
2. **Transformation:** The chunk is passed to the loaded BAAI/bge-m3 model.
3. **Output:** The model outputs a 1024-dimension vector (an array of 1024 numbers) that numerically represents the meaning of that chunk.

This collection of vectors, along with the original text and metadata

2.5 Query Pipeline and Response Generation

Defines the sequence of operations that transforms a user's raw query into a final, cited answer.

The pipeline consists of the following steps:

1. **Query Embedding:** The user's raw query (e.g., "What is the recommended dosage for Metformin?") is passed to the same BAAI/bge-m3 embedding model used during ingestion. This converts the question into a 1024-dimension vector that represents its semantic meaning.
2. **Semantic Search (Retrieval):** This query vector is used to perform a similarity search against the vectors in our Supabase (pgvector) database. The database retrieves the 'top-k' most relevant text chunks whose vectors are numerically closest to the query's vector.
3. **Prompt Augmentation:** A new, detailed prompt is constructed for the generative LLM. This prompt does not just contain the user's question; it is improved with the retrieved text chunks. This provides the LLM with the specific, factual context it needs to form an accurate answer.

Example Augmented Prompt: You are a helpful medical assistant. Using only the context below, answer the user's question. You must cite the source for every claim you make, referencing the "source" and "page".

4. **LLM Response Generation:** This complete, augmented prompt is sent to a generative Large Language Model (LLM), such as a locally-run Llama 3.1 model. The LLM's task is to summarize the provided context and

generate a human-readable answer to the question. Because the prompt instructs it to use only the context, the risk of hallucination is significantly reduced.

5. **Citation Parsing and Formatting:** The raw text response from the LLM is then processed one final time. The system parses this output to extract the answers and the citations, linking them to the original metadata. The final output is then presented to the user via highlighted snippets.

3 Possible Improvements

3.1 Improving Retrieval Accuracy

To ensure the most relevant context is always passed to the LLM, we can implement more advanced retrieval strategies:

- **Hybrid Search:** Combine the current semantic search with traditional keyword-based search, ensuring that queries containing specific, rare keywords are still matched accurately.
- **Reranking:** After the initial retrieval from the vector database, a more powerful but slower cross-encoder model can be used to re-rank the top results.

3.2 Preventing Hallucinations

The RAG architecture is the primary defense against hallucinations by grounding the LLM in factual context. This can be further improved by:

- **Stricter Prompting:** restricting the LLM's prompt and explicitly respond with sentences such as "I do not have enough information" in case the retrieved context does not contain the answer.
- **Citation Verification:** A post-processing step can be added to verify that every statement in the generated answer maps directly back to a retrieved text chunk.

3.3 Scalability, Performance, and Maintainability

To prepare the system for production-level loads and ensure long-term stability, we would focus on:

- **Scalability:** Supabase with pgvector can be scaled by upgrading database resources. For even larger loads, the ingestion and query pipelines can be moved to a distributed architecture using containerization (e.g., Docker) and orchestration (e.g., Kubernetes).

API Layer: A REST API will serve as the primary interface to the RAG system. It will expose an endpoint that accepts a user's question as input. This API layer allows the RAG logic to be integrated into various applications.

- **Performance:** For the LLM, we can utilize optimized inference engines like TensorRT-LLM to achieve higher throughput and lower latency. The embedding and retrieval steps can also be batched to handle concurrent user requests more efficiently.

Caching Strategy: To optimize performance and reduce redundant computations, a caching layer can be added. We will implement a semantic cache. When a new query is received, its embedding is compared against a cache of recent query embeddings. This significantly reduces latency for common questions.

4 Local Prototype

I developed a prototype available in my github at the following link:
RAG System Prototype

Given that the free trial Supabase account provides limited storage, I opted to test the full Retrieval-Augmented Generation (RAG) system locally.

For this purpose, I selected a couple of medical reports to pass through the RAG pipeline. The process was carried out in four main steps: PDF ingestion, chunking, embedding, and querying.

Step 1: PDF Ingestion

The first step involves extracting structured text from the PDF documents.

```
Step 1/2: Running hybrid parsing (PyMuPDF + Tesseract OCR fallback)...
- Page 1 parsed digitally (PyMuPDF).
- Page 2 parsed digitally (PyMuPDF).
- Page 3 parsed digitally (PyMuPDF).
- Page 4 parsed digitally (PyMuPDF).
- Page 5 parsed digitally (PyMuPDF).
- Page 6 parsed digitally (PyMuPDF).
- Page 7 parsed digitally (PyMuPDF).
- Page 8 parsed digitally (PyMuPDF).
- Page 9 parsed digitally (PyMuPDF).
- Page 10 parsed digitally (PyMuPDF).
- Page 11 parsed digitally (PyMuPDF).
- Page 12 parsed digitally (PyMuPDF).
```

Figure 1: Screenshot of the PDF ingestion process.

```
Step 3: Normalizing schema to JSON...
Step 4: Validating key sections...
- No date pattern found.
Step 5: Creating final data object with metadata.
```

Figure 2: Screenshot of the PDF ingestion process.

After ingestion, the extracted content is saved as a JSON file for further processing.

[illegible]

Figure 3: Screenshot of the ingested PDF in json format.

Step 2: Chunking

In this step, the structured text is divided into semantic chunks suitable for embedding and retrieval.

```
Semantic Chunker initialized for BAAI/bge-m3 (Max Tokens: 8192).
Chunking by paragraph, with a fallback window of 500 words.
Starting semantic chunking by PARAGRAPH...
Successfully created 160 semantic chunks.

Created 160 semantic chunks and saved to outputs\chunks_20251020_234706.json.
```

Figure 4: Screenshot of the chunking process.

```
{
  "text": "## title\nExtracted PDF Content",
  "metadata": {
    "source_file": "C:\\Users\\jimo\\Downloads\\HERALD_CV-NCOV-004-Protocol_test.pdf",
    "section": "title",
    "token_count": 9,
    "page": "1",
    "paragraph_index_on_page": 0
  }
},
{
  "text": "## pages\nCLINICAL TRIAL PROTOCOL\nCOVID-19: \\nA Phase 2b/3, Randomized, Observer-Blinded, Placebo-Controlled, Multicenter \\nClinical Study Eva  
Investigational SARS-CoV-2 \\nmRNA Vaccine C\\nCoV in Adults 18 Years of Age and Older \\n\\nProtocol Number: \\n \\nCV-NCOV-004 \\nEudraCT Number: \\n2020-003  
(referred to as C\\nCoV) \\nTrial Name: \\nHERALD \\nPhase: \\nPhase 2b/3 \\nSponsor: \\nCureVac AG \\nSchumannstrasse 27 \\n60325 Frankfurt \\nGermany \\nShort Ti  
\\nProtocol Version: \\n3.0 \\nProtocol Date: \\n29 March 2021 \\n \\nPROTOCOL VERSION HISTORY: \\nVersion 1.0 dated 11 November 2020 \\nVersion 2.0 dated 17 Fe  
or communicated to a third party without the written \\npermission of CureVac AG.",
  "metadata": {
    "source_file": "C:\\Users\\jimo\\Downloads\\HERALD_CV-NCOV-004-Protocol_test.pdf",
    "section": "pages",
    "token_count": 112,
    "page": "1",
    "paragraph_index_on_page": 0
  }
}
```

Figure 5: Screenshot of the chunking JSON output.

Step 3: Embedding

Each chunk is converted into a vector representation using the embedding model, which allows the RAG system to perform semantic retrieval.

```
Loading embedding model: BAAI/bge-m3...
Embedding model loaded successfully.
Embedding batch of 160 chunks...
Batch embedding complete.
Embedded 160 chunks and saved to outputs\embedded_chunks_20251020_234706.json.
```

Figure 6: Screenshot of the embedding process.

```
{
  "text": "## title\nExtracted PDF Content",
  "metadata": {
    "source_file": "C:\\Users\\jimp\\Downloads\\HERALD_CV-NCOV-004-Protocol_test.pdf",
    "section": "title",
    "token_count": 5,
    "page": "1",
    "paragraph_index_on_page": 0
  },
  "vector": [
    0.9020957979275815,
    0.12696565653267988,
    0.055077461965049035,
    0.004728898717348828,
    0.22297446664739573,
    0.49532332170919524,
    0.2578880413765767,
    0.4241954745974149,
    0.1936360204730393,
    0.30921592431185496,
    0.9959537578645501,
    0.8670089354305723,
    0.6926608361907957,
    0.3044249864080677,
    0.3101403831210505,
    0.11724136710752664,
    0.05488548003884508,
    0.740933072687546,
    0.5741861008946594,
    0.023978077114528262
  ]
}
```

Figure 7: Screenshot of the embedded chunks saved as JSON.

Step 4: Query Pipeline

Finally, the system is tested with queries to ensure it can retrieve and summarize information correctly from the embedded chunks. An additional response validator has been added in order to prevent hallucinations.

```
Received query: "Summarize this file"
Step 1: Embedding query...
Step 2: Performing local semantic search (top-k=3)...
Found 3 relevant chunks.
Step 3: Building chat messages...
Step 4: Generating response from LLM...
LLM Raw Response: This file appears to be a clinical trial protocol for a COVID-19 mRNA vaccine, specifically the CV-NCOV-004 study. The protocol outlines the study's objectives, methods, and procedures for conducting the trial.

The study aims to assess the safety and efficacy of the vaccine in preventing COVID-19 in participants. The protocol describes the inclusion and exclusion criteria for participants, the study design, and the procedures for data collection and analysis.

The protocol also outlines the procedures for monitoring and reporting adverse events (AEs), including serious adverse events (SAEs) and medically-attended visits. Participants will be contacted regularly by phone to assess their general well-being and to report any AEs.

The protocol includes definitions for virologically-confirmed COVID-19 cases, which are based on positive SARS-CoV-2 RT-PCR tests in participants with clinically symptomatic disease.

Overall, the protocol provides a comprehensive outline of the study's design, methods, and procedures for conducting the trial and collecting data.
```

Figure 8: Screenshot of the query pipeline in action and the response.

After this step the response of the LLM is validated through a citation check.

```

--- VALIDATOR STEP ---

Verifying citations and factual grounding...
Uncited claim: 'This file appears to be a clinical trial protocol for a COVID-19 mRNA ...'
- Checking uncited claim against ALL context...
[PASSED] 'This file appears to be a clinical trial protocol for a COVI...' - lexical=0.80, semantic=1.00
Uncited claim: 'The protocol outlines the study's objectives, methods, and procedures ...'
- Checking uncited claim against ALL context...
[PASSED] 'The protocol outlines the study's objectives, methods, and p...' - lexical=0.38, semantic=1.00
Uncited claim: 'The study aims to assess the safety and efficacy of the vaccine in pre...'
- Checking uncited claim against ALL context...
[PASSED] 'The study aims to assess the safety and efficacy of the vacc...' - lexical=0.62, semantic=1.00
Uncited claim: 'The protocol describes the inclusion and exclusion criteria for partic...'
- Checking uncited claim against ALL context...
[PASSED] 'The protocol describes the inclusion and exclusion criteria ...' - lexical=0.40, semantic=1.00
Uncited claim: 'The protocol also outlines the procedures for monitoring and reporting...'

```

Figure 9: Screenshot of the validator step.

```

--- Final RAG Output ---
Query: Summarize this file
Answer: This file appears to be a clinical trial protocol for a COVID-19 mRNA vaccine, specifically the CV-NCOV-004 study. The protocol outlines the study's objectives, methods, and procedures for conducting the trial. The study aims to assess the safety and efficacy of the vaccine in preventing COVID-19 in participants. The protocol describes the inclusion and exclusion criteria for participants, the study design, and the procedures for data collection and analysis.

The protocol also outlines the procedures for monitoring and reporting adverse events (AEs), including serious adverse events (SAEs) and medically-attended visits. Participants will be contacted regularly by phone to assess their general well-being and to report any AEs.

The protocol includes definitions for virologically-confirmed COVID-19 cases, which are based on positive SARS-CoV-2 RT-PCR tests in participants with clinically symptomatic disease.

Overall, the protocol provides a comprehensive outline of the study's design, methods, and procedures for conducting the trial and collecting data.

```

Figure 10: Screenshot of RAG final response.

```

Citations saved successfully: outputs\citations_20251020_234706.json
--- Context 1 ---
Source: C:\Users\jimpo\Downloads\HERALD_CV-NCOV-004-Protocol_test.pdf
Page: 7
Text: ## pages
Clinical Trial Protocol

CV-NCOV-004
CureVac AG

COVID-19 mRNA Vaccine
Version 3.0 (29 March 2021)
CONFIDENTIAL
Page 10 of 162
LIST ...
--- Context 2 ---
Source: C:\Users\jimpo\Downloads\HERALD_CV-NCOV-004-Protocol_test.pdf
Page: 104
Text: ## pages
Clinical Trial Protocol

```

Figure 11: Screenshot of citations

This step-by-step approach demonstrates the functionality of the RAG system in a local environment.